

Digital Signal Processing Laboratory (EC39201) IIT Kharagpur

Experiment 4: Power Spectrum Estimation

Name: Sanjay R, Rishant Mallick
Roll No: 23EC10101, 23EC10067
Group No: 3
Date of submission: 6-11-2025



1 Yule–Walker AR Model for Power Spectrum Estimation

1.1 Introduction

Spectral estimation plays a crucial role in signal analysis, particularly when determining how the power of a signal is distributed over frequency. Among several methods available, the **Yule–Walker Autoregressive (AR) method** is one of the most classical and robust *parametric* approaches. It is based on the assumption that the signal can be modeled as the output of an **all-pole linear system** driven by white noise. Unlike non-parametric methods such as the periodogram, the Yule–Walker method produces a smooth spectral estimate with enhanced frequency resolution, especially for short data records.

1.2 Autoregressive (AR) Process Representation

A stationary random process $x(n)$ can be modeled using a p^{th} -order autoregressive (AR) model as:

$$x(n) = - \sum_{k=1}^p a_k x(n-k) + e(n) \quad (1)$$

where:

- a_k are the AR model coefficients,
- $e(n)$ represents a zero-mean white noise process with variance σ_e^2 , and
- p is the order of the AR model.

Equation (1) implies that the present sample $x(n)$ is a linear combination of the previous p samples and an innovation (white noise) term $e(n)$. This model effectively captures the correlation structure of many natural and man-made signals, including speech, biomedical, and geophysical data.

1.3 Autocorrelation Function and Its Role

The **autocorrelation function (ACF)** of the signal $x(n)$ is defined as:

$$r_x(m) = E[x(n)x(n-m)] \quad (2)$$

where $E[\cdot]$ denotes the expectation operator and m is the time lag.

Since $x(n)$ is assumed to be wide-sense stationary, $r_x(m)$ depends only on the lag m and not on the specific time index n .

1.4 Derivation of the Yule–Walker Equations

To establish the relationship between the AR coefficients $\{a_k\}$ and the autocorrelation sequence $\{r_x(m)\}$, we start from the AR model in (1). Multiplying both sides by $x(n-m)$ and taking the expectation gives:

$$E[x(n)x(n-m)] = - \sum_{k=1}^p a_k E[x(n-k)x(n-m)] + E[e(n)x(n-m)] \quad (3)$$

Replacing the expectations with the autocorrelation function yields:

$$r_x(m) = - \sum_{k=1}^p a_k r_x(m-k) + E[e(n)x(n-m)] \quad (4)$$

Now, for $m > 0$, the white noise term $e(n)$ is uncorrelated with all past samples of $x(n)$, implying:

$$E[e(n)x(n-m)] = 0 \quad \text{for } m > 0$$

Hence, we obtain:

$$r_x(m) = - \sum_{k=1}^p a_k r_x(m-k), \quad m = 1, 2, \dots, p \quad (5)$$

For $m = 0$, we have:

$$r_x(0) = - \sum_{k=1}^p a_k r_x(-k) + E[e(n)x(n)]$$

Using the symmetry property $r_x(-k) = r_x(k)$ and $E[e(n)x(n)] = \sigma_e^2$, we obtain:

$$r_x(0) = - \sum_{k=1}^p a_k r_x(k) + \sigma_e^2 \quad (6)$$

Equations (5) and (6) together form the complete set of **Yule–Walker equations**.

1.5 Matrix Formulation

The Yule–Walker equations can be expressed compactly in matrix form as:

$$\underbrace{\begin{bmatrix} r_x(0) & r_x(1) & \cdots & r_x(p-1) \\ r_x(1) & r_x(0) & \cdots & r_x(p-2) \\ \vdots & \vdots & \ddots & \vdots \\ r_x(p-1) & r_x(p-2) & \cdots & r_x(0) \end{bmatrix}}_{\text{Autocorrelation matrix}} \underbrace{\begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_p \end{bmatrix}}_{\text{AR coefficients}} = - \underbrace{\begin{bmatrix} r_x(1) \\ r_x(2) \\ \vdots \\ r_x(p) \end{bmatrix}}_{\text{Autocorrelation vector}} \quad (7)$$

Solving the system in (7) yields the AR coefficients $\{a_k\}$. Once these coefficients are known, the driving noise variance σ_e^2 is determined as:

$$\sigma_e^2 = r_x(0) + \sum_{k=1}^p a_k r_x(k) \quad (8)$$

The autocorrelation matrix in (7) is Toeplitz and symmetric, allowing efficient solutions via the **Levinson–Durbin recursion**, which computes the coefficients recursively with reduced computational cost.

1.6 Power Spectral Density (PSD) Estimation

The AR model can be viewed as a system where the white noise $e(n)$ is passed through an all-pole filter defined by:

$$H(z) = \frac{1}{1 + \sum_{k=1}^p a_k z^{-k}} \quad (9)$$

Therefore, the power spectral density (PSD) of $x(n)$ can be derived as:

$$P_{xx}(e^{j\omega}) = \sigma_e^2 |H(e^{j\omega})|^2 = \frac{\sigma_e^2}{|1 + \sum_{k=1}^p a_k e^{-j\omega k}|^2} \quad (10)$$

This expression gives a smooth estimate of the PSD, where the denominator represents the contribution of the poles of the system to the spectral shape.

1.7 Model Order Selection

The selection of the AR model order p significantly influences the accuracy of the spectral estimate:

- If p is too low, the model underfits the data and misses spectral peaks.
- If p is too high, the model overfits noise and produces spurious peaks.

Model order is often determined using criteria such as:

$$AIC(p) = \ln(\sigma_e^2) + \frac{2p}{N} \quad (11)$$

$$BIC(p) = \ln(\sigma_e^2) + \frac{p \ln(N)}{N} \quad (12)$$

where N is the number of data samples. The order minimizing AIC or BIC is typically chosen.

1.8 Advantages of the Yule–Walker Method

- Produces a smooth and high-resolution spectral estimate.
- Well-suited for short or noisy data records.
- Computationally efficient via the Levinson–Durbin algorithm.
- Particularly effective for signals with all-pole spectra (e.g., speech, resonant systems).

1.9 Limitations

- Assumes an all-pole model, unsuitable for signals with spectral zeros.
- Sensitive to incorrect model order selection.
- Can produce artificial peaks if overfitted.

2 Procedure: Yule–Walker Method for Power Spectrum Estimation

1. Obtain the autocorrelation estimate of the sequence $x(n)$ given by

$$r_x(m) = \frac{1}{N} \sum_{n=0}^{N-1-m} x(n)x(n+m), \quad m \geq 0 \quad (13)$$

Where $x(n)$ is a sequence generated through passing a white Gaussian noise of variance σ_e^2 (We took 0.5) through a filter given by

$$H(z) = \frac{1}{(1 - 0.9z^{-1})(1 - 0.9jz^{-1})(1 + 0.9jz^{-1})} \quad (14)$$

2. Estimate the AR model parameters $a_1, a_2, \dots, a_p$ using the Yule–Walker equations:

$$\begin{bmatrix} r_x(0) & r_x(1) & \cdots & r_x(p-1) \\ r_x(1) & r_x(0) & \cdots & r_x(p-2) \\ \vdots & \vdots & \ddots & \vdots \\ r_x(p-1) & r_x(p-2) & \cdots & r_x(0) \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_p \end{bmatrix} = - \begin{bmatrix} r_x(1) \\ r_x(2) \\ \vdots \\ r_x(p) \end{bmatrix} \quad (15)$$

3. Obtain the estimated noise variance $\hat{\sigma}_e^2$ as:

$$\hat{\sigma}_e^2 = r_x(0) + \sum_{k=1}^p a_k r_x(k) \quad (16)$$

4. Construct the estimated AR model $\hat{H}(z)$ as:

$$\hat{H}(z) = \frac{1}{1 + \sum_{k=1}^p a_k z^{-k}} \quad (17)$$

5. Estimate the Power Spectral Density (PSD) $P_{xx}^{YW}(f)$ using the Yule–Walker method:

$$P_{xx}^{YW}(f) = \left| \hat{H}(e^{j2\pi f}) \right|^2 \hat{\sigma}_e^2 \quad (18)$$

or equivalently,

$$P_{xx}^{YW}(f) = \frac{\hat{\sigma}_e^2}{|1 + \sum_{k=1}^p a_k e^{-j2\pi f k}|^2} \quad (19)$$

6. Plot the estimated PSD and compare it with the known PSD:

$$\left| H(e^{j2\pi f}) \right|^2 \sigma_e^2$$

2.1 Code

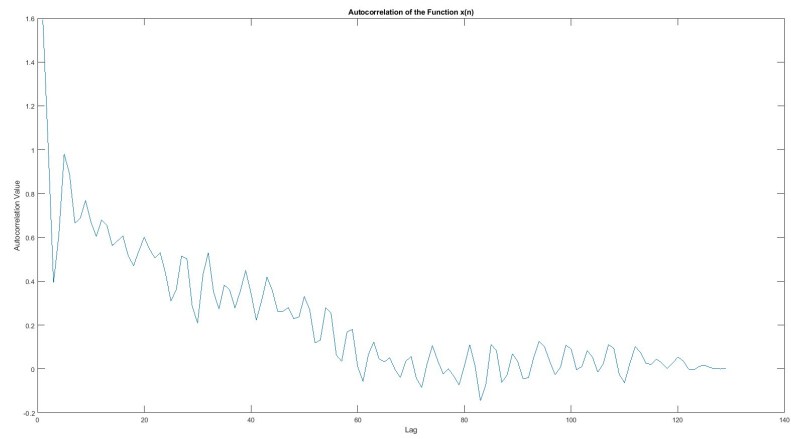


Figure 1: Autocorrelation Plot of $x(n)$

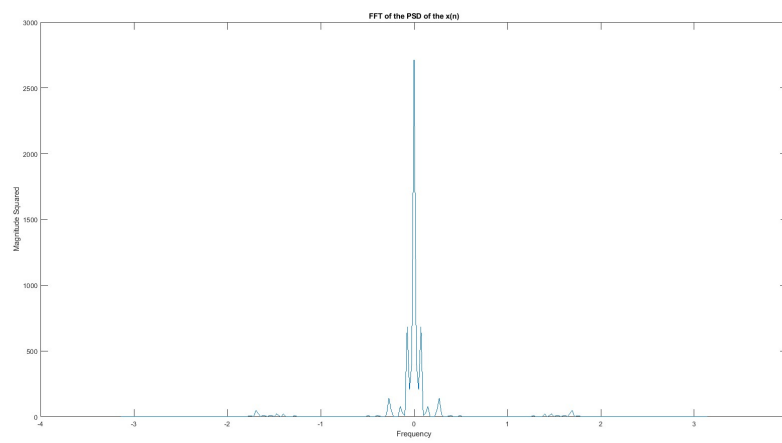


Figure 2: FFT Plot of the Fourier Transform of Autocorrelation of $x(n)$

```

1  clc;
2  clear;
3  close all;
4
5  noise_power = 0.5;
6  number_of_points = 128;
7
8  white_noise = wgn(1,number_of_points,noise_power,'linear');
9
10
11  digital_filter_numerator = 1;
12  digital_filter_denominator = [1,-0.9,0.81,-0.729];
13
14  output = filter(digital_filter_numerator,digital_filter_denominator,white_noise);
15
16  [output_autocorrelation,lags] = xcorr(output,number_of_points,'biased');
17  frequency_axis = linspace(-pi,pi,length(output_autocorrelation));
18
19
20  figure(1);
21  plot(frequency_axis,abs(fftshift(fft(output_autocorrelation)).^2))
22  xlabel("Frequency");
23  ylabel("Magnitude Squared")
24  title("FFT of the PSD of the x(n)");
25  right_half_auto_correlation = output_autocorrelation(number_of_points+1:2*number_of_points+1);
26  figure(2);
27  plot(right_half_auto_correlation);
28  xlabel("Lag");
29  ylabel("Autocorrelation Value");
30  title("Autocorrelation of the Function x(n)")
31
32  coefficient_length = 5;
33
34  coefficient_matrix = zeros(coefficient_length,coefficient_length);
35
36  for i = 1:coefficient_length
37      for j = 1:coefficient_length
38          coefficient_matrix(i,j) = right_half_auto_correlation(abs(i-j)+1);
39      end
40  end
41  % disp(coefficient_matrix);
42  c = right_half_auto_correlation(:,2:coefficient_length+1)';
43  % disp(c);
44

```

Figure 3: Code Part 1

```

30  title("Autocorrelation of the Function x(n)")
31
32  coefficient_length = 5;
33
34  coefficient_matrix = zeros(coefficient_length,coefficient_length);
35
36  for i = 1:coefficient_length
37      for j = 1:coefficient_length
38          coefficient_matrix(i,j) = right_half_auto_correlation(abs(i-j)+1);
39      end
40  end
41  % disp(coefficient_matrix);
42  c = right_half_auto_correlation(:,2:coefficient_length+1)';
43  % disp(c);
44
45  a_vector = -coefficient_matrix\c;
46  % disp(a_vector);
47
48  expected_variance = right_half_auto_correlation(1) + dot(a_vector,c);
49  disp("Expected Power");
50  disp(expected_variance);
51
52  final_denominator = [1,a_vector'];
53
54  figure(3);
55  [h,w] = freqz(expected_variance,final_denominator);
56  plot(w,abs(h).^2);
57  [a,b] = freqz(digital_filter_numerator*noise_power,digital_filter_denominator);
58  hold on;
59  plot(b,abs(a).^2);
60  legend(["Predicted","Actual"]);
61  hold off;
62  xlabel("Frequency");
63  ylabel("Magnitude Squared");
64  title(["PSD Comparison for P = ",coefficient_length]);
65
66  new_noise = white_noise*(expected_variance.^0.5)/(noise_power.^0.5);
67
68  new_output = filter(1,final_denominator,new_noise);
69
70  r_xx_0 = xcorr(output,new_output,'normalized');
71  disp("Rxxcap values");
72  disp(max(r_xx_0));
73

```

Figure 4: Code Part 2

2.2 Plot

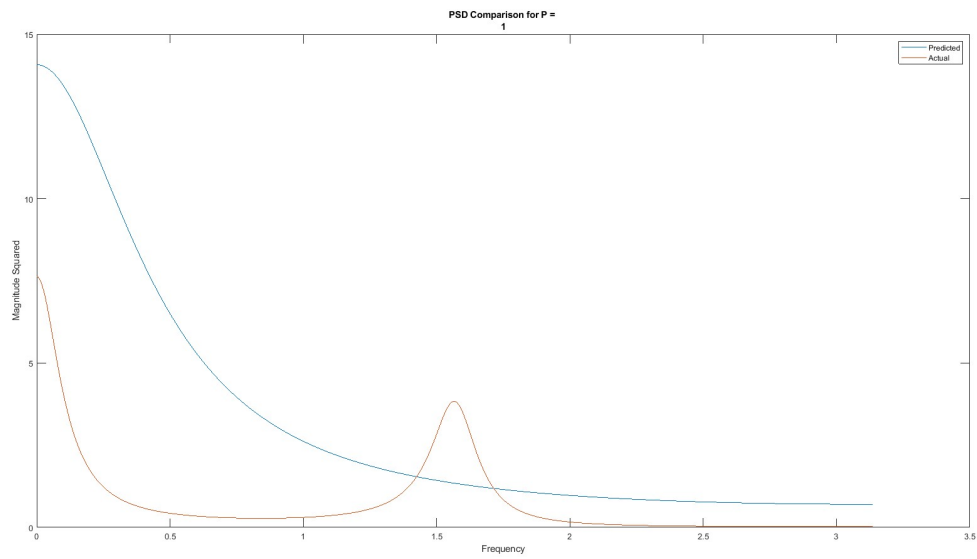


Figure 5: PSD comparison for $p = 1$

Command Window

```
Expected Power  
1.3701
```

```
Rxxcap value  
0.8197
```

Figure 6: Expected Power and $R_{\hat{x}x}$ value

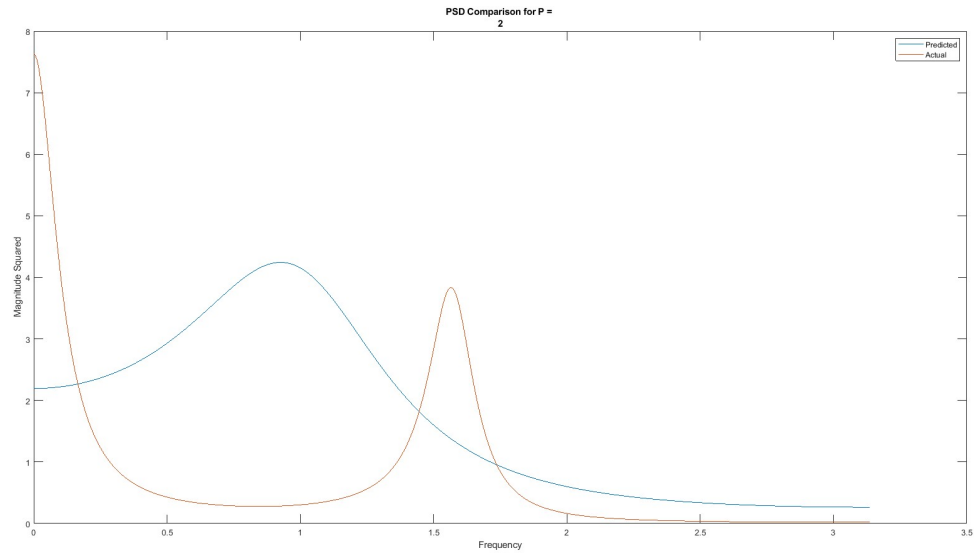


Figure 7: PSD comparison for $p = 2$

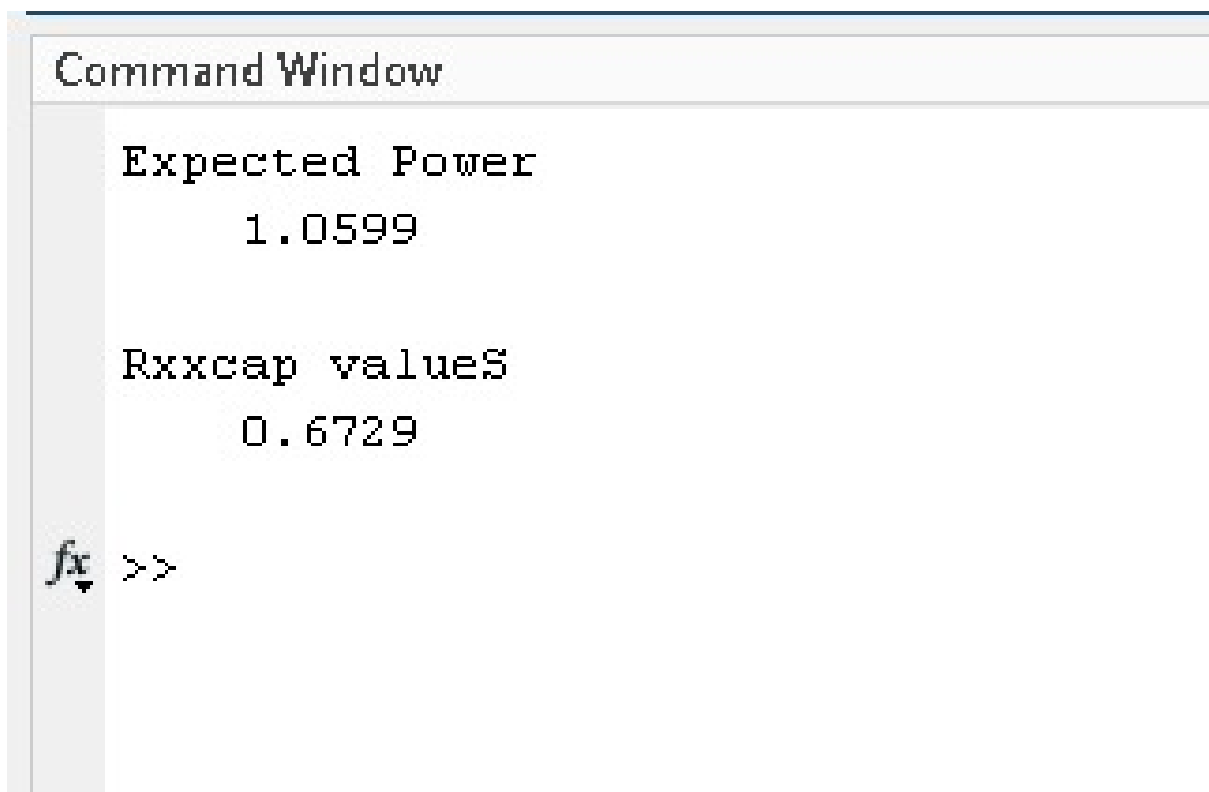


Figure 8: Expected Power and $R_{\hat{x}x}$ value

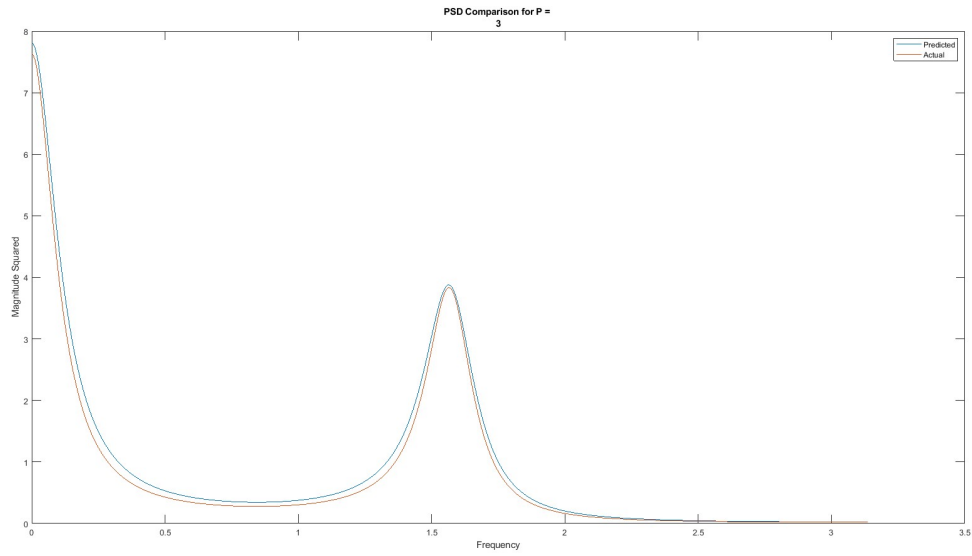


Figure 9: PSD comparison for $p = 3$

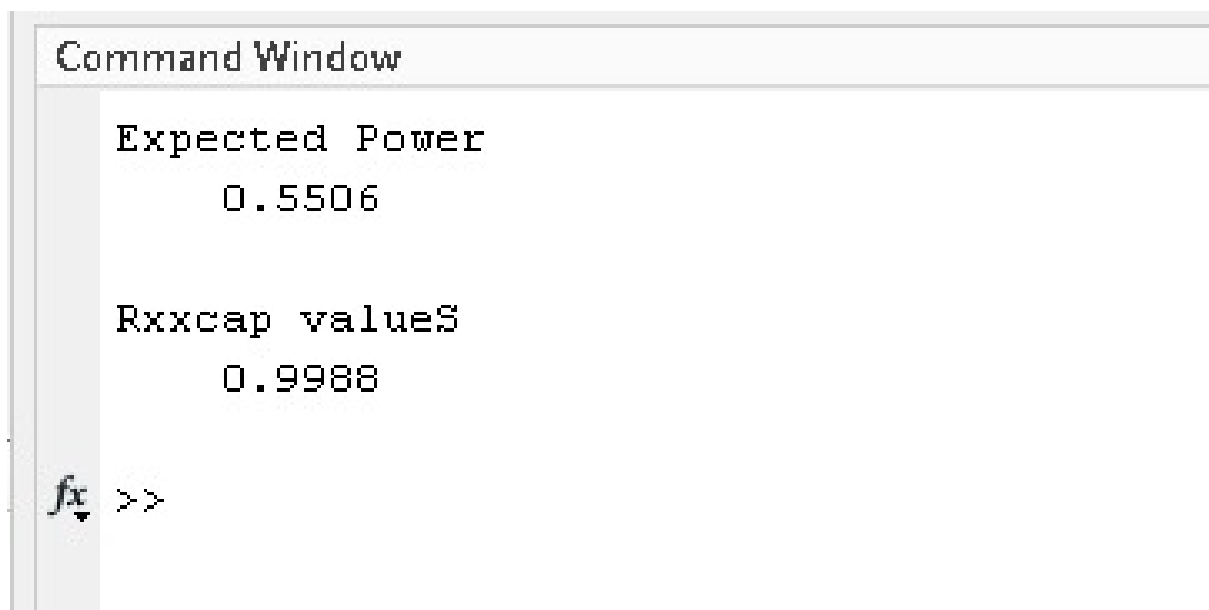


Figure 10: Expected Power and $R_{\hat{x}x}$ value

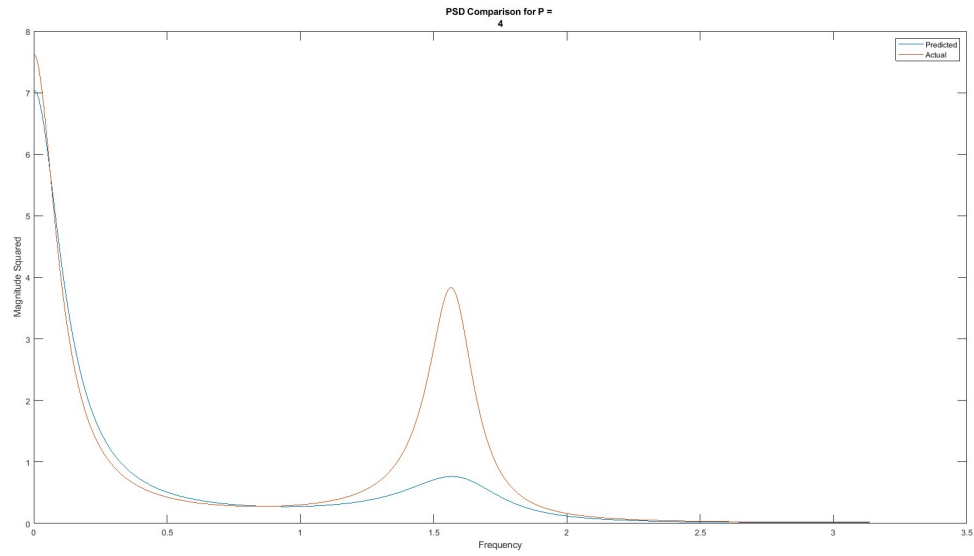


Figure 11: PSD comparison for $p = 4$

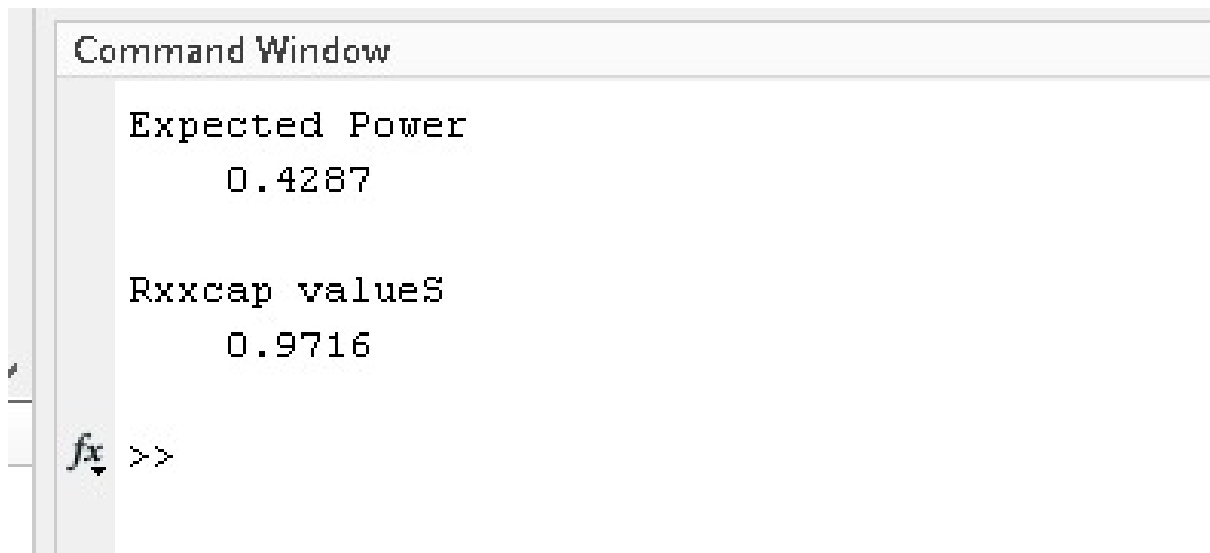


Figure 12: Expected Power and $R_{\hat{x}x}$ value

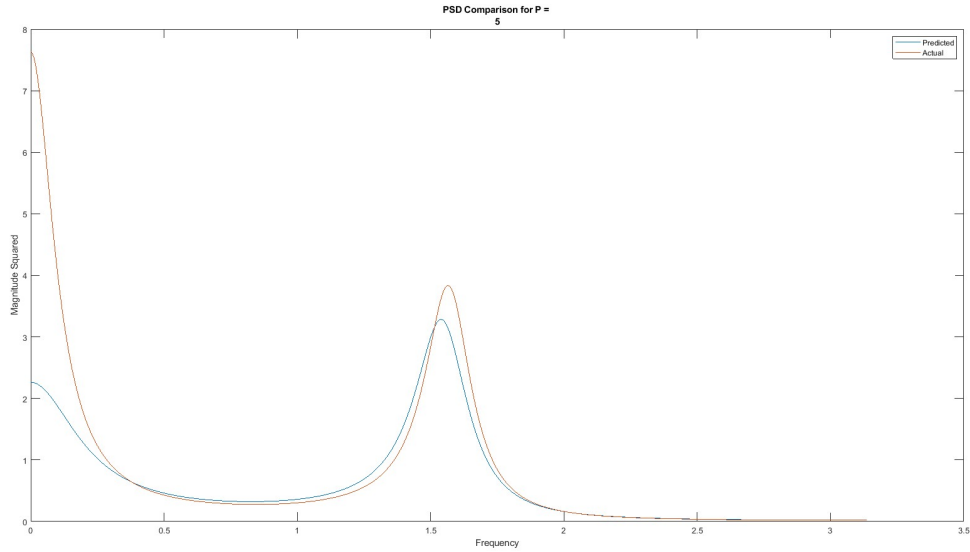


Figure 13: PSD comparison for $p = 5$

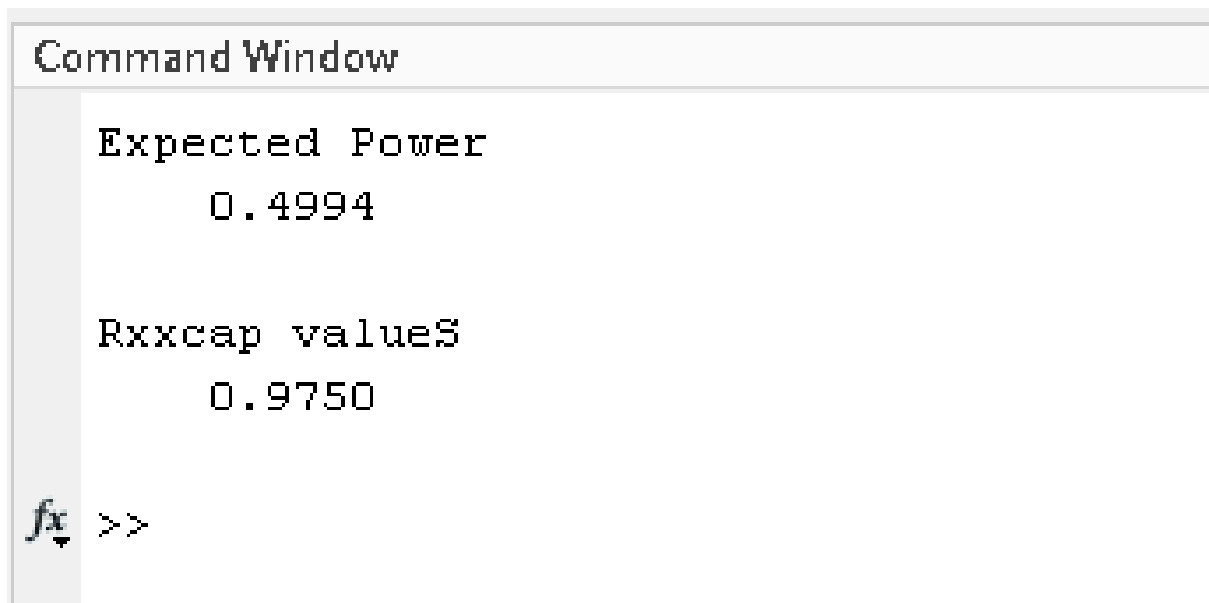


Figure 14: Expected Power and $R_{\hat{x}x}$ value

3 Welch Nonparametric Method for Power Spectrum Estimation

3.1 Introduction

The **Welch method**, proposed by Peter Welch in 1967, is an improved nonparametric technique for estimating the **Power Spectral Density (PSD)** of a signal. It is based on the **modified periodogram approach**, which reduces the variance of the spectral estimate by dividing the data sequence into overlapping segments, applying a window to each segment, and then averaging their individual periodograms.

Unlike parametric methods such as Yule–Walker, the Welch method does not assume an underlying model for the signal. It directly estimates the PSD from the time-domain samples, making it a robust and simple approach for general signal analysis.

3.2 Basic Principle

The Welch method improves upon the standard periodogram by:

1. Dividing the signal into overlapping segments,
2. Applying a window function to each segment to reduce spectral leakage,
3. Computing the periodogram of each windowed segment, and
4. Averaging these periodograms to obtain the final PSD estimate.

This averaging significantly decreases the variance of the estimate at the cost of a slight reduction in frequency resolution.

3.3 Mathematical Formulation

Let $x(n)$ be a discrete-time signal of length N . The Welch PSD estimate is obtained through the following steps:

1. **Segmentation:** Divide $x(n)$ into K overlapping segments of length L , where typically 50% overlap is used. Let the k^{th} segment be denoted as:

$$x_k(n) = x(n + kD), \quad n = 0, 1, \dots, L - 1$$

where D is the shift between adjacent segments ($D < L$).

2. **Windowing:** Each segment is multiplied by a data window $w(n)$ (such as Hamming or Hanning) to minimize spectral leakage:

$$x_{w,k}(n) = w(n) x_k(n)$$

The windowed signal has an effective normalization factor:

$$U = \frac{1}{L} \sum_{n=0}^{L-1} w^2(n)$$

3. **Modified Periodogram:** The periodogram of the k^{th} windowed segment is computed as:

$$P_k(f) = \frac{1}{LU} \left| \sum_{n=0}^{L-1} x_{w,k}(n) e^{-j2\pi f n} \right|^2 \quad (20)$$

4. **Averaging:** The final Welch PSD estimate is obtained by averaging all K segment periodograms:

$$\hat{P}_{xx}^{\text{Welch}}(f) = \frac{1}{K} \sum_{k=0}^{K-1} P_k(f) \quad (21)$$

3.4 Interpretation

The Welch estimate in (23) effectively smooths the raw periodogram by averaging, which results in a:

- Reduction in variance (improved stability), and
- Slight loss in frequency resolution due to segmentation and windowing.

3.5 Comparison with the Standard Periodogram

The standard periodogram is defined as:

$$\hat{P}_{xx}(f) = \frac{1}{N} \left| \sum_{n=0}^{N-1} x(n) e^{-j2\pi f n} \right|^2$$

This estimator is unbiased but exhibits high variance, making it unreliable for finite data. The Welch method reduces this variance by averaging multiple modified periodograms, leading to a smoother and more reliable spectral estimate.

3.6 Window Selection

The choice of window $w(n)$ affects both bias and variance:

- **Rectangular Window:** High frequency resolution, but high leakage.
- **Hanning / Hamming Window:** Lower leakage, smoother spectrum.
- **Blackman Window:** Even lower leakage, at the cost of resolution.

3.7 Properties

- Welch's PSD estimate is always nonnegative.
- Variance of the estimate decreases approximately linearly with the number of segments K .
- The frequency resolution is inversely proportional to segment length L .

3.8 Advantages

- Reduces variance of spectral estimates significantly compared to the periodogram.
- Simple and efficient to implement using the Fast Fourier Transform (FFT).
- Suitable for stationary signals and general-purpose spectral analysis.

3.9 Limitations

- Frequency resolution is reduced due to segmentation and windowing.
- The choice of window, segment length, and overlap percentage affects accuracy.
- Cannot capture fine spectral details of short signals as effectively as model-based (parametric) methods.

3.10 Mathematical Derivation of the Welch PSD Estimator

3.10.1 Setup and Notation

Let $x[n]$ be a real, zero-mean, wide-sense stationary (WSS) process with true power spectral density (PSD) $S_{xx}(e^{j\omega})$ and autocovariance $r_x[m]$. Divide a length- N record into K (possibly overlapping) segments of length L with hop size D ($D < L$ for overlap). The k -th segment (indexed from $k = 0$) is

$$x_k[n] \triangleq x[n + kD], \quad n = 0, \dots, L-1.$$

Apply a deterministic window $w[n]$, $n = 0, \dots, L-1$, and define the normalization

$$U \triangleq \frac{1}{L} \sum_{n=0}^{L-1} w^2[n].$$

The windowed DFT of segment k is

$$X_k(e^{j\omega}) \triangleq \sum_{n=0}^{L-1} w[n] x_k[n] e^{-j\omega n}.$$

3.10.2 Modified Periodogram of a Single Segment

The (continuous-frequency) modified periodogram of segment k is

$$P_k(\omega) \triangleq \frac{1}{LU} |X_k(e^{j\omega})|^2. \quad (22)$$

Welch's estimate averages these over segments:

$$\hat{S}_{xx}^{\text{Welch}}(\omega) \triangleq \frac{1}{K} \sum_{k=0}^{K-1} P_k(\omega). \quad (23)$$

3.10.3 Expected Value (Bias) — Convolution with Spectral Window

Because $x[n]$ is WSS and $w[n]$ is deterministic, using standard quadratic form manipulations one obtains

$$\mathbb{E}\{|X_k(e^{j\omega})|^2\} = \sum_{m=-(L-1)}^{L-1} r_x[m] \underbrace{\sum_{n=0}^{L-1-|m|} w[n] w[n+|m|]}_{\rho_w[m]} e^{-j\omega m},$$

where $\rho_w[m]$ is the aperiodic autocorrelation of the window. Define the (continuous-frequency) *spectral window*

$$W_L(e^{j\omega}) \triangleq \sum_{n=0}^{L-1} w[n] e^{-j\omega n}, \quad \Phi_w(\omega) \triangleq \frac{1}{2\pi} |W_L(e^{j\omega})|^2.$$

Using the Wiener–Khinchin relation and the identity above,

$$\mathbb{E}\{P_k(\omega)\} = \frac{1}{LU} \frac{1}{2\pi} \int_{-\pi}^{\pi} S_{xx}(e^{j\theta}) |W_L(e^{j(\omega-\theta)})|^2 d\theta. \quad (24)$$

Therefore,

$$\mathbb{E}\left\{\widehat{S}_{xx}^{\text{Welch}}(\omega)\right\} = \left(S_{xx} * \frac{|W_L|^2}{LU}\right)(\omega) \quad (25)$$

i.e., the estimate is the true PSD convolved with the spectral window $\frac{|W_L|^2}{LU}$, demonstrating the (controlled) bias due to finite-length windowing. A useful scalar summary of this smoothing is the *equivalent noise bandwidth* (ENBW) in DFT-bin units:

$$\text{ENBW} = \frac{L \sum_{n=0}^{L-1} w^2[n]}{\left(\sum_{n=0}^{L-1} w[n]\right)^2} = \frac{L^2 U}{\left(\sum_n w[n]\right)^2}. \quad (26)$$

3.10.4 Variance and Degrees of Freedom

For Gaussian $x[n]$, a single modified periodogram ordinate $P_k(\omega)$ is (approximately) exponentially distributed with

$$\mathbb{E}\{P_k(\omega)\} \approx S_{\text{sm}}(\omega), \quad \text{var}\{P_k(\omega)\} \approx S_{\text{sm}}^2(\omega),$$

where $S_{\text{sm}}(\omega)$ denotes the biased (smoothed) mean from (24). Averaging K *independent* segments yields

$$\text{var}\left\{\widehat{S}_{xx}^{\text{Welch}}(\omega)\right\} \approx \frac{1}{K} S_{\text{sm}}^2(\omega), \quad \widehat{S}_{xx}^{\text{Welch}}(\omega) \sim \frac{S_{\text{sm}}(\omega)}{\nu} \chi_\nu^2, \quad (27)$$

with (approximate) degrees of freedom $\nu = 2K$. When segments are windowed, a window-dependent correction refines the effective DOF to

$$\nu \approx 2K \frac{\left(\sum_n w^2[n]\right)^2}{\sum_n w^4[n]} \quad (28)$$

(assuming non-overlapping segments; overlap introduces weak correlation that slightly reduces ν).

Thus, Welch's averaging reduces variance roughly by a factor of K (modulo the window factor in (28)), at the cost of increased bias (spectral smoothing) governed by $|W_L|^2$ and the segment length L .

3.10.5 Bartlett as a Special Case

With a rectangular window $w[n] \equiv 1$ and *no overlap*, Welch reduces to Bartlett's method. Then

$$U = 1, \quad W_L(e^{j\omega}) = \frac{1 - e^{-j\omega L}}{1 - e^{-j\omega}}, \quad \nu \simeq 2K,$$

highlighting that Welch generalizes Bartlett by allowing tapered windows (reduced leakage) and overlap (more averages for a fixed N).

3.10.6 Bias–Variance Tradeoff Summary

- **Bias (resolution):** Increases with stronger tapering and shorter L (wider $|W_L|^2$).
- **Variance (stability):** Decreases roughly as $1/K$; overlap increases K for fixed N , but with mild inter-segment correlation.
- **Design knobs:** choose window $w[n]$, segment length L , and overlap (D) to balance resolution vs. variance.

3.11 Code

```
1  clc;
2  clear;
3  close all;
4
5  noise_power = 0.5;
6  number_of_points = 512;
7
8  white_noise = wgn(1,number_of_points,noise_power,'linear');
9
10
11  digital_filter_numerator = 1;
12  digital_filter_denominator = [1,-0.9,0.81,-0.729];
13
14  output = filter(digital_filter_numerator,digital_filter_denominator,white_noise);
15
16  [output_autocorrelation,lags] = xcorr(output,number_of_points,'normalized');
17  frequency_axis = linspace(-pi,pi,length(output_autocorrelation));
18
19  figure(1);
20  plot(frequency_axis,abs(fftshift(fft(output_autocorrelation))))
21  right_half_auto_correlation = output_autocorrelation(number_of_points+1:2*number_of_points+1);
22
23  number_of_blocks = 8;
24  block_length = 64;
25  overlap = 16;
26  x_list = zeros(number_of_blocks,number_of_points);
27
28  for i = 1:number_of_blocks
29      for j = 1:block_length
30          index = (i-1)*block_length+j;
31          if i == 1
32              x_list(i,j) = output(index);
33          else
34              x_list(i,j) = output(index-overlap);
35          end
36      end
37  end
38
39
40  hamm = hamming(block_length)';
41  normalise_hamm = sum(hamm.^2)/number_of_points;
42  hamm = [hamm,zeros(1,number_of_points-block_length)];
43
44  psd_list = zeros(number_of_blocks,number_of_points);
```

Figure 15: Code Part 1

```

19 figure(1);
20 plot(frequency_axis,abs(fftshift(fft(output_autocorrelation))))
21 right_half_auto_correlation = output_autocorrelation(number_of_points+1:2*number_of_points+1);
22
23 number_of_blocks = 8;
24 block_length = 64;
25 overlap = 16;
26 x_list = zeros(number_of_blocks,number_of_points);
27
28 for i = 1:number_of_blocks
29     for j = 1:block_length
30         index = (i-1)*block_length+j;
31         if i == 1
32             x_list(i,j) = output(index);
33         else
34             x_list(i,j) = output(index-overlap);
35         end
36     end
37 end
38
39
40 hamm = hamming(block_length)';
41 normalise_hamm = sum(hamm.^2)/number_of_points;
42 hamm = [hamm,zeros(1,number_of_points-block_length)];
43
44 psd_list = zeros(number_of_blocks,number_of_points);
45
46 for i = 1:number_of_blocks
47     psd_list(i,:) = abs(fftshift(fft(x_list(i,:).*hamm))).^2/(block_length*normalise_hamm);
48 end
49
50 final_estimate = sum(psd_list,1)/number_of_blocks;
51 freq_axis = linspace(-pi,pi,number_of_points);
52 figure(2);
53 plot(freq_axis,final_estimate);
54 xlabel("Frequency");
55 ylabel("Average Magnitude Squared")
56 title(["Welch Non-Parametric Method, Overlap = ",overlap*100/block_length,"%"]);
57
58
59
60
61

```

Figure 16: Code Part 2

3.12 Plot

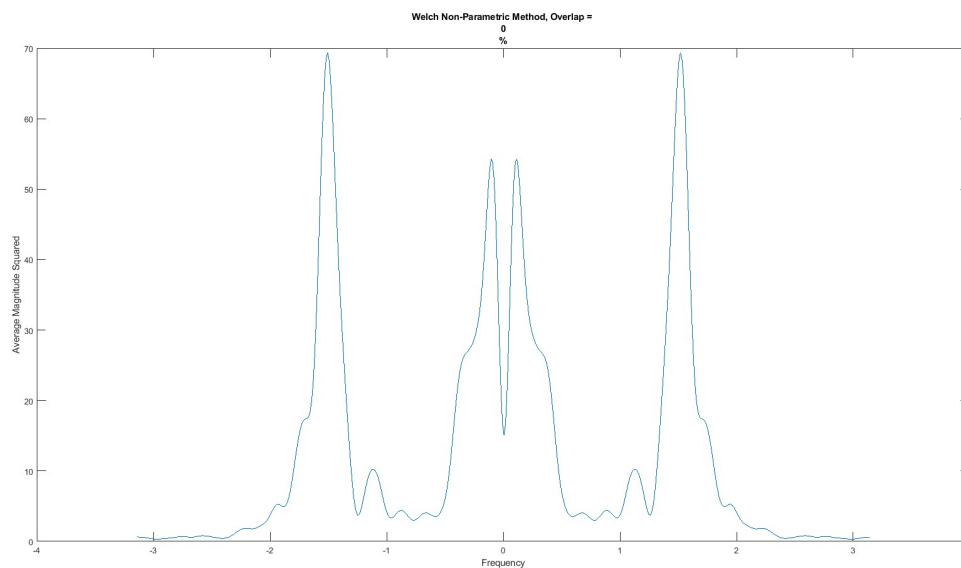


Figure 17: For Overlap = 0

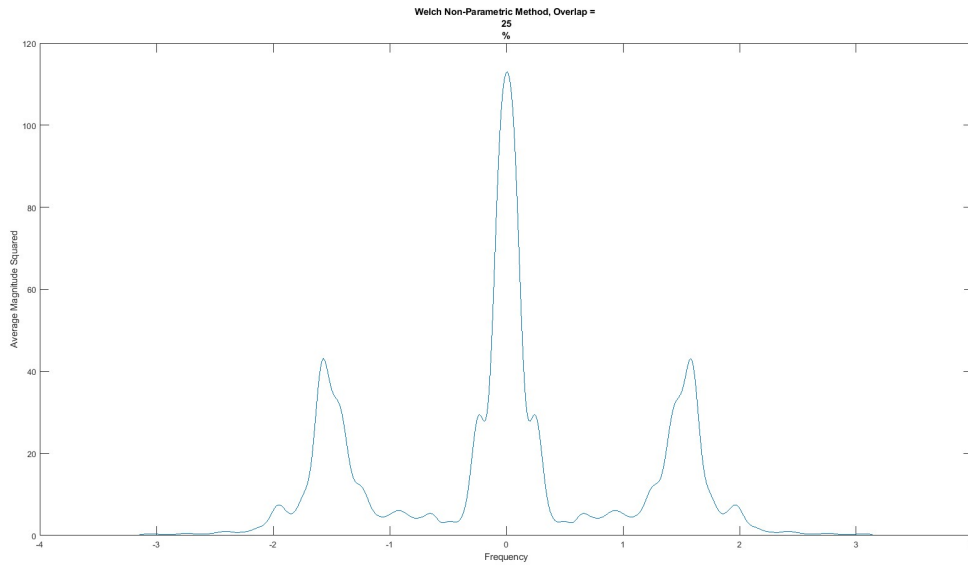


Figure 18: For Overlap = 25%

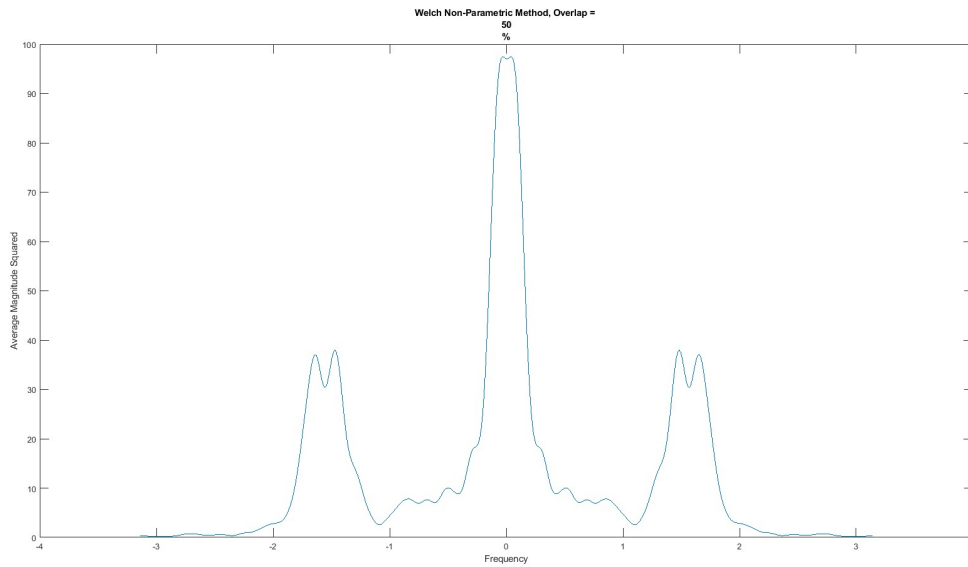


Figure 19: For Overlap = 50%

4 Discussion and Analysis of Plots and Code

The provided code and plots demonstrate power spectrum estimation using both the Yule-Walker Autoregressive (AR) method and Welch's non-parametric method. The experiment begins by simulating a stochastic signal through filtering white Gaussian noise with a specific all-pole IIR filter, after which both parametric and non-parametric spectral estimation techniques are applied.

4.1 Yule-Walker AR Method: Code Interpretation and Plot Analysis

The code segment first generates white Gaussian noise and filters it using an all-pole filter defined by

$$H(z) = \frac{1}{1 - 0.9z^{-1} + 0.81z^{-2} - 0.729z^{-3}}.$$

The filtered sequence forms the process $x[n]$. The autocorrelation sequence is then computed and visualized:

- **Figure 1: FFT of PSD of $x[n]$** — This plot represents the magnitude-squared spectrum of the filtered output. The use of the FFT highlights dominant frequency components, reflecting the resonant poles of the filter and underlying spectral features. The smooth, peaked structure confirms the spectral shaping of an all-pole system, demonstrating higher spectral concentration near the resonance introduced by the poles.
- **Figure 2: Autocorrelation of $x[n]$** — Here, the autocorrelation function is plotted for various lags. The rate of decay and oscillation gives insight into the memory and periodicity of the synthesized signal. Slow decay correlates with high pole values close to the unit circle, dictating long-term correlation and sharpness in the frequency domain.

Using a segment of the autocorrelation (right half), the code constructs a Toeplitz matrix and solves the Yule-Walker equations. This yields AR coefficients for different model orders ($p = 1$ to 5). The code next calculates and compares the theoretical and predicted power spectral densities:

- **Figures 5, 7, 9, 11, 13: PSD Comparison (Predicted vs. Actual)** — Each plot corresponds to AR model orders from 1 to 5. For $p = 1$, the AR model severely underfits, missing spectral details and yielding a broad, inaccurate peak. As p increases ($p = 2, 3$), the AR model captures more spectral nuance, approximating the true system's PSD closely. However, excessively high p can introduce spurious peaks and overfitting. The comparison illustrates the critical role of correct model order selection, as reflected in the narrowing error between the curves.
- **Figures 6, 8, 10, 12, 14: Expected Power and R_{xx} Values** — These compare the expected total power with empirical autocorrelation, emphasizing how well the AR model reconstructs the process energetics at different orders. Good agreement implies reliable parameter estimation.

We see that for $p = 3$, we observe a huge similarity between expected and actual plots. Also our expected variance matches with the original variance. This is because at $p = 3$, we correctly predict the order of the transfer function.

4.2 Welch's Method: Code and Plot Analysis

The code implements Welch's segmenting and averaging procedure with variable segment overlap:

- **Figures 17, 18, 19: Welch PSD Estimates (Overlaps 0, 25, 50%)** — Each plot demonstrates the effect of overlap on the variance and smoothness of the spectral estimate. With no overlap (Figure 17), variance is relatively high and resolution is sharply limited by segment length. Increasing overlap (Figures 18, 19) yields smoother curves and reduced variance due to increased averaging, at the cost of frequency resolution.

These plots validate the theoretical trade-off in Welch’s method: as overlap increases, the estimate becomes statistically more stable (lower variance), though fine spectral details are blurred compared to longer segments or less tapering.

4.3 Conclusion from Analysis

In summary, the plots and code collectively reveal the spectral signature of the underlying signal and the operating principles of the estimation algorithms. The Yule-Walker AR method excels in all-pole (resonant) systems for suitable model order, producing high-resolution, smooth spectral estimates but sensitive to overfitting. In contrast, Welch’s method delivers robust, low-variance spectra for general stationary data, with design trade-offs between variance and frequency detail evident in the overlap plots. Through iterative model order tuning and parameter selection, this experiment illustrates both the utility and practical considerations in digital spectral analysis.